GENERALITAT VALENCIANA
Conselleria d'Educació, Universitats i Ocupació

ceedcv CENTRE ESPECÍFIC D'EDUCACIÓ A DISTÀNCIA DE LA COMUNITAT VALENCIANA

Unitat 04 — Estructures de control i excepcions bàsiques
Bloc 02

1r CFGS DAW · Programació

Sergi García Barea

Excepcions bàsiques

La idea en una frase

> Una excepció és un **objecte que Java llança quan alguna cosa eix malament**, i que hereta de `Throwable`. El teu programa pot estar o no preparat per a atrapar-lo: aprendre a llegir estos crits és aprendre a depurar.

T'ha passat que un programa es "cau" amb un munt de text roig? Eixe text és una excepció — i és or: et diu **què**, **on** i en **quin mètode**.

El crash: la teua primera excepció

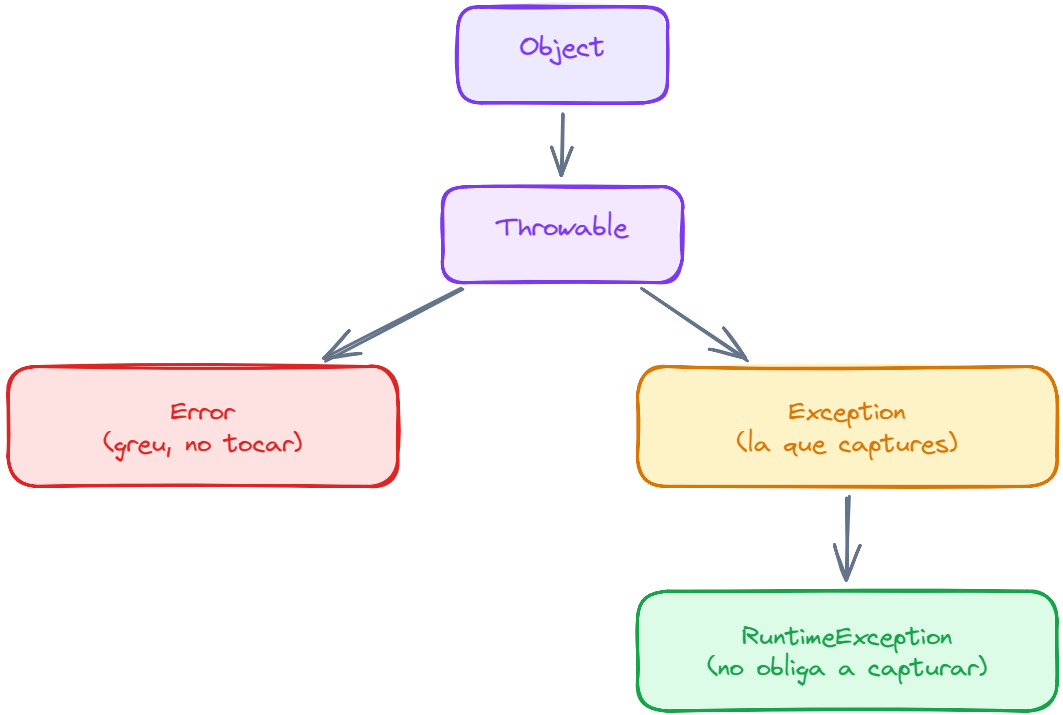
Explosio.java

```java
public class Explosio {
    public static void main(String[] args) {
        int[] numeros = { 1, 2, 3 };
        System.out.println(numeros[5]); // no existeix!
    }
}
```

```
$ java Explosio Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException:
Index 5 out of bounds for length 3 at Explosio.main(Explosio.java:5)
```

ℹ️ Llig la primera línia: el nom de l'excepció et diu **què** ha passat, i la línia amb `at ...` et diu **on**. Un GPS amb acusacions.

L'arbre genealògic de Throwable



Ací viuen: ArithmeticException, NullPointerException, NumberFormatException...

La família Throwable: detalls

RuntimeException típiques

```
int x = 10 / 0;                    // ArithmeticException
a[9] = 1;  // array de 3:          // ArrayIndexOutOfBounds
String s = null; s.length();       // NullPointerException
Integer.parseInt("Hola");          // NumberFormatException
```

ℹ️ **Error:** problemes greus de la JVM — ignora'ls. **Exception:** ací viu el 99% de la teua vida.

⚠️ **RuntimeException** = es llança en **temps d'execució**, no en compilar. El compilador no t'avisa — i no t'obliga a capturar-la.

Les més comunes: la guia de camp

Excepció	Quan apareix
ArithmeticException	Dividir entre 0
ArrayIndexOutOfBoundsException	Índex fora de l'array
NullPointerException	Tocar un objecte null
NumberFormatException	Convertir text que no és nombre
InputMismatchException	Scanner rep el tipus equivocat

⚠️ La **NullPointerException** és el clàssic absolut de Java: apareix en tocar un objecte que no existia. La teua àvia, si programara, també la tindria.

Mini-chequeig: excepcions

🧠 El detectiu d'excepcions — quina llança cada línia?

```
int a = 5 / 0;
String[] dies = {"L", "M", "X"};
System.out.println(dies[3]);
String text = null;
System.out.println(text.toUpperCase());
int b = Integer.parseInt("quaranta-dos");
```



🔄 **Respostes:**

- **ArithmeticException** — divisió entre zero
- **ArrayIndexOutOfBoundsException** — un array de 3 s'indexa 0, 1, 2
- **NullPointerException** — text val null · **NumberFormatException** — eixe text no és un nombre

try, catch i finally

La idea en una frase

> `try` protegix el codi perillós, `catch` atrapar l'excepció si apareix i `finally` s'executa **sempre**, ocorrega el que ocorrega. És l'airbag del codi: sense ell, la primera InputMismatchException mata el programa.

try/catch: el programa sobreviu

LectorBlindat.java

```
try {
    System.out.print("Quants anys tens? ");
    int edat = sc.nextInt();
    System.out.println("Vas nàixer fa " + edat + " anys.");
} catch (InputMismatchException e) {
    System.out.println("Això no és un nombre. No em faces això.");
}
System.out.println("El programa seguix viu. 🎉");
```

✅ Amb **try/catch** el catch atrapar l'error, imprimeix un missatge i **el programa continua**.

⚠️ **Catch buit = pecat mortal**: et tragues l'error ni adonar-te'n. Com a mínim, imprimeix un missatge.

Captura múltiple i finally

🎯 Diversos catch en fila

```
try {
    // codi perillós
} catch (ArithmeticException e) {
    // més específic primer
} catch (RuntimeException e) {
    // general al final
}
```



```
<div class="warning" style="margin-top:0.5rem;">
  <strong>Del més específic al més general</strong>, com els else if. Un <code>catch (Exception e)</code> al principi es menja les esp
</div>
```

Exemple guiat: el menú a prova de bombes

MenuBlindat.java

```
do {
    System.out.println("1. Jugar  2. Eixir");
    try {
        opcio = sc.nextInt();
        valida = (opcio == 1 || opcio == 2);
        if (!valida) System.out.println("Opció no vàlida.");
    } catch (InputMismatchException e) {
        System.out.println("Això no és un nombre.");
        sc.next(); // descarta el text brossa del buffer
    }
} while (!valida);
```

ℹ️ Fixa't en `sc.next()` dins del catch: sense ell, la porqueria seguix al buffer del Scanner i el següent `nextInt()` tornaria a fallar. Atrapar l'excepció i **netejar el buffer** són dos passos del mateix ball.

Mini-chequeig: try/catch

🧠 El detectiu de l'ordre — què passa ací?

```
try {
    int[] numeros = {10, 20};
    System.out.println(numeros[3]);
} catch (Exception e) {
    System.out.println("Atrapat per Exception.");
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Atrapat per l'específica.");
}
```

📋

🔄 **Resposta:**

- **No compila**: el segon catch és inalcançable perquè `Exception` ja atrapar tot — subclasse davant del pare

throw i excepcions pròpies

La idea en una frase

> `throw` llança una excepció **quan tu decideixes** que alguna cosa no ha de continuar, i creant la teua pròpia excepció (heretant de `RuntimeException` o `Exception`) pots posar-li **nom propi** al problema.

throw: llança la pedra

CompteBancari.java

```
double saldo = 10.0;
double retir = 500.0;

if (retir > saldo) {
    throw new ArithmeticException(
        "Saldo insuficient: " + saldo);
}

saldo -= retir;
```

⚠️ **throw ≠ throws:** `throw` **llança** (al cos del mètode); `throws` **anuncia** en la signatura que el mètode pot llançar excepcions checked.

ℹ️ El missatge del constructor és el que veuràs amb `e.getMessage()`. Una excepció primerenca val més que un bug dues setmanes després.

Excepció pròpia a mida

ProducteEsgotatException.java

```java
public class ProducteEsgotatException extends RuntimeException {
    public ProducteEsgotatException(String producte) {
        super("El producte " + producte + " està esgotat.");
    }
}
```

MaquinaExpenedora.java

```java
try {
    if (stock == 0) {
        throw new ProducteEsgotatException("Refresc");
    }
    System.out.println("Ací tens el teu Refresc");
} catch (ProducteEsgotatException e) {
    System.out.println("Ho sentim: " + e.getMessage());
}
```

checked vs unchecked

ℹ️ **Checked (controlades)**: el compilador **t'obliga** a capturar-les o declarar-les amb `throws`. Hereten d' `Exception` però no de RuntimeException. Exemple: `IOException`. Si no les gestiones, **no compila**.

✅ **Unchecked (no controlades)**: no t'obliguen a res. Són `RuntimeException` i les seues filles: ArithmeticException, NullPointerException, NumberFormatException...

ℹ️ Hereta d'**Exception** si vols **obligar** a capturar la teua excepció; de **RuntimeException** si prefereixes que no els obligue. Per a començar, RuntimeException és més còmoda.

Mini-chequeig: throw

🧠 El controlador de notes — nota = 15:

```
public class NotaInvalidaException extends RuntimeException {
    public NotaInvalidaException(double nota) {
        super("La nota " + nota + " no està entre 0 i 10.");
    }
}
// al main: if (nota < 0 || nota > 10) { throw new NotaInvalidaException(nota); }
```

📋

🔄 **Resposta:**

- El programa es talla amb **NotaInvalidaException: La nota 15.0 no està entre 0 i 10.**
- Hereta de RuntimeException (→ Exception → Throwable): tot el comportament de llançar-se i capturar-se, el constructor amb `super(missatge)` i `getMessage()`

Repàs: controla les estructures

El joc de les decisions

🎮 Tria saviament:

1. nota = 6 amb `if (nota ⩾ 5) → "Aprovat"` primer... què imprimeix?
2. Quantes voltes fa `for (int i = 0; i < 3; i++)`?
3. case 1 i case 2 seguits sense break, variable = 1: què passa?

🔄 **Solucions:**

- **"Aprovat"**: guanya el primer if true — l'ordre mana, encara que 6 no arribe a 7
- **3 voltes**: i val 0, 1 i 2 — amb < 3 mai no entra amb i = 3
- **Fall-through**: el case 1 es desborda al case 2

Fireside chat: if-else vs switch

ℹ️ **if-else:** — Jo soc el clàssic. Condicions, rangs, comparacions... necessites decidir si és més gran que 5 o està entre 10 i 20? Crida'm. Jo compare el que siga.

ℹ️ **switch:** — I tu t'omplires d'else if fins que el codi sembla l'escala d'un edifici. Amb mi poses la variable una vegada i cada cas en la seua línia.

⚠️ **if-else:** — Elegant fins que t'oblides un break i el teu switch es converteix en un tobogan.
switch: — El fall-through s'usa a propòsit per a agrupar casos. I tu, amb trenta else if, saps quin va abans que quin?

✅ **La lliçó:** no hi ha guanyador. switch per a **valors concrets**, if/else if per a **rangs i mescles**. Triar bé és la mitat de l'examen.

Qui soc?

🕵️ Endevina quin concepte de la unitat soc:

1. **Soc el semàfor**: si la meua condició és true, deixe passar; si és false, redirigix a l'altre carril.
2. **Soc el menú del restaurant**: mires la meua variable i executes el case que coincidisca.
3. **Soc la cinta que comprova abans de córrer**: si la condició és false d'entrada, no faig ni un pas.
4. **Soc el botó de parada**: talle el bucle sencer tan bon punt aparec.
5. **Soc l'avi de tots els errors**: tot el que es llança hereta de mi.
6. **Soc l'airbag**: atrapar l'error perquè el programa no muera.

🔄 **Respostes:**

- 1 → **l'if/else** · 2 → **el switch** · 3 → **el while** (el do-while corre primer)
- 4 → **el break** · 5 → **Throwable** · 6 → **el catch**

Resum del bloc 02

Tres frases per quedar-te



Excepcions

Objectes que hereten de Throwable. Llig el missatge: què, on i en quin mètode. RuntimeException es llança en executar i no obliga a capturar.



try/catch/finally

try protegix, catch atrapa (de l'específic al general, mai buit) i finally s'executa sempre: ideal per a tancar recursos.



throw i pròpies

throw llança quan tu decideixes; throws declara. Les teues excepcions hereten de RuntimeException o Exception i porten nom propi.

// EDUCACIÓN A DISTANCIA // EDUCACIÓN PÚBLICA Y GRATUITA // EDUCACIÓN DE CALIDAD // EDUCACIÓN AUTONÓMICA //